Universidad de Buenos Aires
Facultad De Ingeniería

Taller de Comunicaciones Digitales

Simulación y Análisis de un Sistema de Comunicaciones

1er cuatrimestre 2026

Simulación y analisis en forma integral un sistema de comunicaciones capaz de transmitir texto de un extremo a otro. Se abarcan temas como la codificación de fuente, las modulaciones en banda base y en banda pasante, los efectos del canal (ruido, ISI) y los procesos en el receptor.

Nombre y Apellido	**Padrón**	**Correo Electrónico**
Berard, Lucia Magdalena	101213	lberard@fi.uba.ar
Stejman Peterburg, Julián	102840	jstejman@fi.uba.ar

Índice

1. Introducción **2**

2. Desarrollo **3**
2.1. Módulo A: Programa principal – Datos, control y estructura general del sistema 3
2.2. Módulo B: Codificación y decodificación de fuente (Huffman) 4
2.2.1. Resultados 4
2.3. Módulo C: Modulación y demodulación en banda base o pasante 7
2.3.1. Resultados 7
2.4. Módulo D: Efectos del canal 11
2.4.1. Resultados 11
2.5. Módulo E: Codificación de canal (códigos lineales de bloques) 12
2.5.1. Resultados 12

3. Análisis del sistema **15**
3.1. Resultados 15
3.1.1. Probabilidad de Error de Símbolo (P_e) - Sin Codificación de Canal 15
3.1.2. Probabilidad de Error de Bit (P_b) - Sin Codificación de Canal 16
3.1.3. Comparación entre Modulaciones Eficientes en Ancho de Banda vs Energía 17
3.1.4. Efecto de la Codificación de Canal 18
3.1.5. Efecto en el texto ingresado 19

4. Conclusiones **20**

1. Introducción

En este trabajo práctico se buscará simular un sistema de comunicaciones digitales, analizando todos los bloques que atraviesa un texto desde transmisor hacia receptor. El trabajo será implementado en *Python*, aprovechando su ecosistema científico para procesamiento numérico y visualización.

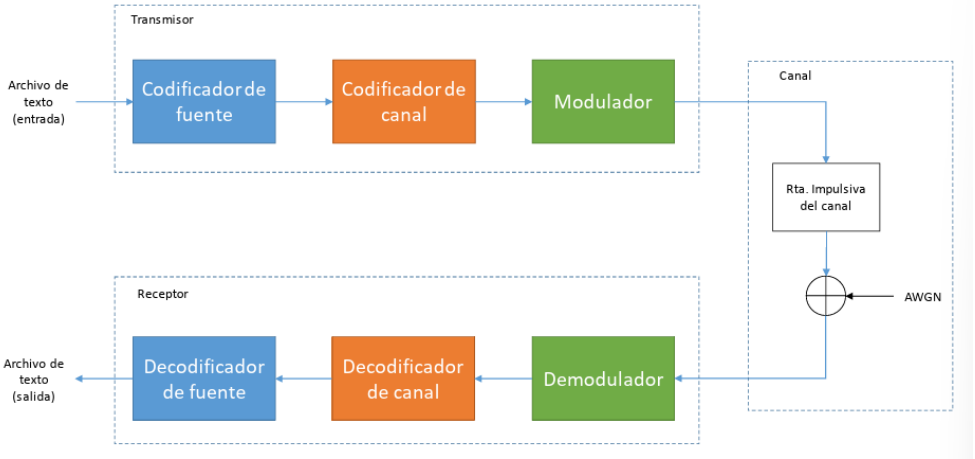


Figura 1.1: Diagrama de bloques de un sistema de comunicaciones

A continuación, se da una breve descripción de cada uno de estos bloques.

- **Codificador de fuente**: elimina redundancias en la entrada para reducir la tasa de símbolos a transmitir y disminuir el ancho de banda necesario. En este caso, se simulará el algoritmo de Huffman para obtener un código óptimo para la representación de los caracteres de texto del archivo que se introducen como entrada. Se busca representar los caracteres con la menor cantidad de bits posible.
- **Codificador de canal**: agrega redundancias de forma controlada para la detección y corrección de errores.
- **Modulador**: transforma los símbolos digitales en formas de onda compatibles con el canal.
- **Canal**: medio físico de transmisión; introduce atenuación, distorsión y ruido.
- **Demodulador**: asigna símbolos a las señales recibidas, según el criterio usado en el transmisor.
- **Decodificador de canal**: utiliza la redundancia agregada en el transmisor para detectar y/o corregir errores.
- **Decodificador de fuente**: reconstruye el mensaje original para recuperar el archivo de texto.

2. Desarrollo

2.1. Módulo A: Programa principal – Datos, control y estructura general del sistema

En este módulo se elabora el programa principal que articula el flujo *Transmisor–Canal–Receptor* y centraliza el ingreso de datos y parámetros de simulación. El programa principal permite:

- Especificar la ubicación y el nombre del archivo de texto a procesar y transmitir.
- Definir los parámetros de control del transmisor, del canal y del receptor.
- Orquestar la invocación de las funciones de cada etapa hasta obtener el archivo recibido y compararlo con el original.

El sistema se estructuró en la siguiente estructura de archivos:

```
tp-ta137/
├── README.md
├── requirements.txt
├── data/
│   ├── input/
│   └── output/
└── src/
    ├── main.py ..................................... Orquestrador CLI
    ├── pipeline.py ................................ Estructura base del pipeline
    ├── file.py ....................................... Lectura y escritura de archivos
    ├── source.py ..................................... (Huffman) Módulo B
    ├── modulation.py ............................... (Modulaciones) Módulo C
    ├── channel.py ................................... (AWGN, atenuación) Módulo D
    ├── cod_channel.py ............................... Módulo E
    ├── viz.py ........................................ Gráficos
    └── utils.py ...................... Utilidades mínimas (BER, RNG, etc.)
```

Donde podemos dividirlo en las cuatro secciones que definen el proyecto:

- Datos y control: Se ingresa el archivo de texto y se fijan todos los parámetros (esquema y orden de modulación, potencia de ruido, etc.). Esta definida por los archivos `main.py` y `file.py`, donde la el archivo inicial se buscarán en `tp-ta137/input/` y el resultado se guardará en `tp-ta137/output/`
- Transmisor: Se implementan el Codificador de fuente, el Codificador de canal y el Modulador. Esta se encuentra dada por los archivos `source.py` encargada de la codificación de fuente, `modulation.py` encargada de la modulación y `cod_channel.py` para la codificación de canal.
- Canal: Se modela el ruido AWGN y la respuesta impulsiva. La simulación del canal se encuentra en el archivo `channel.py`.
- Receptor: Se implementan el Demodulador, el Decodificador de canal y el Decodificador de fuente. Por como se tienen que comunicar entre la codificación y la decodificación se encuentran nuevamente en los archivos `source.py`, `modulation.py`, y `cod_channel.py`.

2.2. Módulo B: Codificación y decodificación de fuente (Huffman)

En este módulo se implementa la codificación de fuente mediante Huffman, desde la estimación de probabilidades y entropía hasta la reconstrucción del texto a partir de las palabras de código.

- **Transmisor**
  1. Se implementa una función que lee un archivo `.txt`, procesa carácter por carácter (letras mayúsculas/minúsculas, dígitos, especiales y de control) y devuelve el vector de probabilidades de aparición de cada carácter.
  2. Se calcula la entropía del texto a partir de dichas probabilidades.
  3. Se construye el código binario de Huffman: función que, dadas las probabilidades, genera la tabla carácter $\leftrightarrow$ palabra de código.
  4. Se calculan la longitud mínima y la longitud promedio del código y se las compara.
  5. Se codifica el texto usando la tabla de Huffman, devolviendo el vector (o flujo) de palabras binarias correspondiente.
- **Receptor**
  1. Se decodifican las palabras de código recibidas y se devuelve el vector de caracteres.
  2. Se genera el archivo de texto de salida a partir del vector de caracteres.

2.2. Resultados

Se prueba el funcionamiento del código con el texto **ADBADEDBBDD** como entrada. Para cada carácter, según su cantidad de apariciones se estima su probabilidad de ocurrencia. Esto se utiliza para la codificación por Huffman, que busca un código óptimo con la menor longitud promedia dada la distribución de probabilidades de los símbolos. Siempre al símbolo más frecuente se le asigna el código más corto, y al símbolo menos frecuente el código más largo. El algoritmo asigna a cada carácter, o símbolo, una palabra de código. Se compara la asignación realizada por la implementación en *Python* con la teórica.

Los caracteres, para el texto particular que se toma como entrada, son A, B, D y E. A partir de la cantidad de veces que aparece cada caracter en el texto, se puede estimar sus probabilidades de ocurrencia. Esta forma de estimación de probabilidades a partir del método de Monte Carlo considera, por ejemplo, para la probabilidad de ocurrencia del caracter A:

$$P(S = A) = \mathbb{E}[\mathbb{1}_{\{S=A\}}] \xrightarrow{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}_{\{c_i=A\}}$$

Donde S es la variable aleatoria correspondiente al símbolo de fuente, la función $\mathbb{1}_{\{\cdot\}}$ corresponde a la función indicadora, c_i es cada carácter emitido en el texto y n es la cantidad total de muestras, de los caracteres emitidos. El texto utilizado fue solamente para probar el funcionamiento de la implementación de esta sección en el código. Es importante mencionar que utilizar tan pocos caracteres resulta en una estimación de probabilidades que pueden no ser representativas de la realidad. Este método de estimación de probabilidades es más eficiente mientras más muestras se toman, como bien se observa en la ecuación de más arriba.

De esta forma, considerando un total de 11 caracteres se tiene para cada símbolo:

Caracter	Cantidad de ocurrrencias	Probabilidad de ocurrencia
D	5	$0,455$
B	3	$0,272$
A	2	$0,182$
E	1	$0,090$

Cuadro 2.1: Tabla de valores de probabilidad de cada carácter para el texto utilizado de ejemplo.

Estos valores coinciden con los obtenidos en la simulación. En la tabla ya se ordenan los caracteres por su probabilidad de ocurrencia decreciente. Utilizando el método de Huffman, se toman los dos símbolos de

menor probabilidad de ocurrencia y se suman estas probabilidades para nuevamente reordenarlas de forma decreciente. Se toman de dos particularmente porque se utiliza un código binario (con dos símbolos). Este proceso se realiza hasta que queden finalmente dos probabilidades, que deben sumar 1. La construcción del código se puede visualizar como un diagrama de árbol o por columnas como se muestra en la figura 2.1.

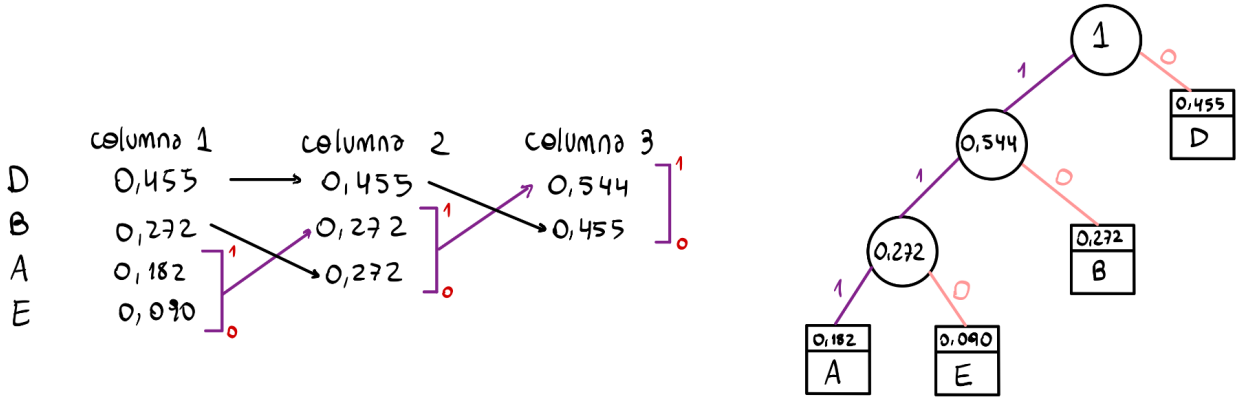


Figura 2.1: Diagrama de construcción del código óptimo por Huffman.

El código obtenido para cada carácter coincide con el de la simulación y se resume en la tabla 2.2 con la longitud de cada código. Se agrega además una comparación con el código de longitud fija ASCII, observando que utiliza una mayor longitud para cada carácter, 7 bits/símbolo de fuente. Utilizando Huffman la longitud del código es variable porque depende de su probabilidad de ocurrencia, haciéndolo más eficiente.

Caracter	Probabilidad de ocurrencia	Código	Longitud código [bits]	Código ASCII
D	$0,455$	0	1	1000100
B	$0,272$	10	2	1000010
A	$0,182$	111	3	1000001
E	$0,090$	110	3	1000101

Cuadro 2.2: Tabla de valores de la codificación de cada carácter para el texto utilizado de ejemplo.

Luego, con la codificación de cada carácter, se codifica el archivo de texto de entrada resultando en: 11101011101100101000. Asimismo, cuando se decodifica se recupera el texto original ADBADEDBBDD.

El resultado de utilizar el método de Huffman para la codificación del archivo de texto es lograr el código más óptimo posible, utilizando la menor cantidad de bits posibles. La longitud promedia de dígitos por símbolo fuente se calcula como:

$$\bar{L} = \sum_{k=0}^{3} p_k \cdot l_k \tag{2.1}$$

Siendo p_k la probabilidad de ocurrencia de cada carácter y l_k la longitud de la palabra de código asociada a cada carácter. La sumatoria se hace sobre cada carácter (por eso es hasta 3). Utilizando el método de Huffman, particularmente se logra que $\bar{L}$ sea mínima. Por el primer teorema de Shannon, se conoce que el valor mínimo teórico de $\bar{L}$ es:

$$L_{\min} = \frac{H(\mathcal{S})}{\log_2(r)} = H(\mathcal{S}) \tag{2.2}$$

Con r la cantidad de símbolos utilizados para construir las palabras de código (2 en este caso) y $H(\mathcal{S})$ la entropía de la fuente con alfabeto $\mathcal{S}$ (conjunto de caracteres). Esta es el valor medio de la información, asociada a los símbolos de la fuente y se calcula como:

$$H(\mathcal{S}) = \sum_{k=0}^{3} p_k \log\left(\frac{1}{p_k}\right) \tag{2.3}$$

Dada cierta distribución de probabilidades, la entropía fijará el valor mínimo que puede tomar $\bar{L}$ y a partir de esto podremos medir la eficiencia del código como:

$$\eta = \frac{L_{\min}}{\bar{L}} = \frac{H(\mathcal{S})}{\bar{L}} \tag{2.4}$$

Idealmente, uno busca que la eficiencia sea 1, indicando un código absolutamente óptimo o adaptado a la fuente. Sin embargo, para ciertas distrribuciones de probabilidad esto no es posible. Esto se debe a que $l_k = -\log_2(p_k)$ puede no ser un número entero. Por esto se definen los códigos quasi absolutamente óptimos. El método de Huffman permite obtener el código de menor longitud promedio posible, que como se mencionó, puede no cumplir con el ideal. Además, el código resultante es un código prefijo, es decir, ninguna palabra de código es prefijo de ninguna otra. Como todo código prefijo es unívocamente decodificable, con el método de Huffman también se cumplirá esta característica. Esto implica que toda secuencia codificada finita, corresponde a una única secuencia de la fuente, lo cual es esencial para poder recuperar el texto original mediante la decodificación.

Se calculó para este caso la entropía de la fuente, la longitud mínima (coincidente con la entropía), la longitud promedio y la eficiencia del código, resumidos en la tabla 2.3.

Entropía de la fuente [bits]	$1,788$
Longitud mínima [bits/símbolo fuente]	$1,788$
Longitud promedio [bits/símbolo fuente]	$1,815$
Eficiencia del código [%]	$98,51$
Eficiencia del código ASCII [%]	$25,54$

Cuadro 2.3: Tabla de valores de entropía, longitud y eficiencia del código.

Todos estos valores se calcularon también por simulación. Se puede observar que no coincide la longitud promedio con la mínima porque en este caso no es posible. Coincidir con la longitud mínima implicaría que la longitud para cada carácter sea:

$$\begin{cases} l_D = 1,14 \text{ bits} \\ l_B = 1,88 \text{ bits} \\ l_A = 2,46 \text{ bits} \\ l_E = 3,47 \text{ bits} \end{cases}$$

Esto no es posible porque la longitud debe ser un número entero. Sin embargo, con Huffman se logró el código más cercano al óptimo posible. Como la longitud promedio está entre la mínima y la mínima más 1, será quasi-absolutamente óptimo.

2.3. Módulo C: Modulación y demodulación en banda base o pasante

En este módulo se implementan y analizan los esquemas de modulación/demodulación seleccionados, junto con la estimación de energías y la visualización de constelaciones y regiones de decisión.

- **Transmisor**

  1. Se implementa una función que recibe un vector binario y modula según el esquema indicado desde el programa principal y el orden $M \in \{2, 4, 8, 16\}$:
     - M-PSK
     - M-FSK

     Etiquetado: Gray o binario, donde se utiliza código de Gray para M-PSK y binario para M-FSK. Los símbolos se representan por sus coordenadas en una base de funciones ortonormales (no por forma de onda). Se adoptan convenciones de energía media de bit unitaria.
  2. Se implementa una función para estimar las energías medias de símbolo y de bit a partir del vector de símbolos modulados.
  3. Desde el programa principal, se grafican la constelación, las regiones de decisión y el etiquetado.

- **Receptor**

  1. Se implementa una función que, dados los símbolos modulados, el esquema y orden M, y el etiquetado, demodula y obtiene la secuencia binaria.
  2. Se implementa una función que compara símbolos transmitidos vs. demodulados para estimar P_e de **símbolo**.
  3. Se implementa una función que compara bits transmitidos vs. recibidos para estimar P_b de **bit**.

2.3. Resultados

La modulación en banda pasante resulta ser necesaria para adaptar la señal que buscamos transmitir al canal. En particular, el espectro de la señal cuando hablamos de banda pasante se ubica a altas frecuencias, centrado en la frecuencia de la señal portadora. Esto es necesario, por ejemplo, cuando se utilizan antenas, cuyo tamaño es proporcional a la longitud de onda y por ende inversamente proporcional a la frecuencia de la señal. Existen varios esquemas de modulación en banda pasante, entre los cuales se encuentra M-ASK, M-PSK, M-QAM y M-FSK. Se toma una señal portadora y en cada esquema se manipulan algunas de sus características: amplitud, fase o frecuencia para transmitir cada símbolo. En el caso de M-ASK se manipula la amplitud de la portadora, para cada símbolo entonces se envía la misma señal pero con distinta amplitud. En M-PSK se manipula la fase de la portadora y en M-FSK la frecuencia. Por último, en M-QAM se manipula tanto la magnitud como la fase. En este trabajo práctico, se analizan dos esquemas de modulación particulares: M-PSK y M-FSK.

Se considera que la señal asociada a cada símbolo es $s_i(t)$. En el caso de M-PSK, se considera que cada señal es de la forma:

$$s_i(t) = \sqrt{\frac{2\epsilon_s}{T_s}} \cos(2\pi f_p t + \phi_i) \qquad i = 0, \ldots, M-1 \;\text{ y }\; 0 \leq t \leq T_s$$

Donde M es la cantidad de símbolos, T_s es la duración de cada símbolo, ϵ_s es la energía de $s_i(t)$ y f_p es la frecuencia de la portadora. Para cada símbolo lo único que cambia es la fase de esta señal. Para el análisis se considera la base ortonormal conformada por las siguientes dos funciones:

$$\begin{cases} \psi_0(t) = \sqrt{\frac{2}{T_s}} \cos(2\pi f_p t) \\ \psi_1(t) = \sqrt{\frac{2}{T_s}} \sin(2\pi f_p t) \end{cases}$$

Ambas de duración T_s. Como estas funciones conforman una base ortonormal del espacio, toda función se podrá escribir como combinación lineal de ellas. Utilizando propiedades trigonométricas se puede reescribir $s_i(t)$ como:

$$s_i(t) = \sqrt{\frac{2\epsilon_s}{T_s}}\left(\cos(2\pi f_p t)\cos(\phi_i) - \sin(2\pi f_p t)\sin(\phi_i)\right)$$

$$s_i(t) = \sqrt{\epsilon_s}\cos(\phi_i)\psi_0(t) - \sqrt{\epsilon_s}\sin(\phi_i)\psi_1(t)$$

Los símbolos se representan a partir de sus vectores de coordenadas en esta base ortonormal. Esto se observa en el gráfico de constelaciones, donde todos los vectores tendrán la misma amplitud pero varía la fase según el símbolo que se transmita. Este gráfico se muestra más adelante en la figura 2.2b. Se puede observar que hay símbolos más cerca entre sí, los vecinos y en el caso de tomar una decisión al observar una señal recibida y cometer un error es probable que sea con un símbolo vecino. Por esta razón, el etiquetamiento en M-PSK se realiza utilizando el código de Gray. Los símbolos vecinos con este etiquetado solo difieren en un bit, por lo que se si llega a cometer un error de símbolo, el error de bit será solo de uno.

En el caso de FSK, las señales de cada símbolo tienen la forma:

$$s_i(t) = \sqrt{\frac{2\epsilon_s}{T_s}}\cos(\omega_i t + \phi)$$

De duración T_s, donde para cada símbolo cambia la frecuencia. En este caso, la base ortonormal tomando $N = 2$ tiene la forma:

$$\begin{cases} \psi_0 = \sqrt{\frac{2}{T_s}}\cos(n\omega_p t) \\ \psi_1 = \sqrt{\frac{2}{T_s}}\cos(m\omega_p t) \end{cases}$$

Las funciones que conforman la base difieren en frecuencia (y son de duración T_s). Cada $s_i(t)$ coincide con una función de la base. Por eso, siempre se da que $M = N$. En este caso, los símbolos están a igual distancia entre sí, no hay símbolos más cercanos como pasaba con M-PSK, por lo que utilizar códigos de Gray para el etiquetamiento pierde sentido. Por eso, en el caso de M-FSK se utilizan códigos binarios.

Para cada esquema de modulación se realiza una tabla donde se expresa la energía media por símbolo y por bit y la probabilidad de error de símbolo y de bit.

La energía media por símbolo se puede calcular como:

$$\bar{\epsilon_s} = \sum_{i=0}^{M-1} p_i \epsilon_{s_i} = \epsilon_s = ||s_i||^2$$

Considerando a los símbolos equiprobables y todos con igual energía. Esto se puede calcular directamente desde los gráficos de constelación en función de la distancia entre símbolos. Como cada símbolo tiene k bits con $k = \log_2(M)$, la energía por símbolo se puede relacionar con la energía por bit como:

$$\bar{\epsilon_s} = \log_2(M) \cdot \bar{\epsilon_b}$$

Cuando el símbolo se recibe, no se recibe el mismo símbolo que se transmitió, dado que el canal puede atenuar la señal y además se suma un ruido blanco gaussiano. Cuando se grafica el símbolo recibido en la constelación, con esta adición de ruido puede no caer exactamente en un símbolo pero la decisión se toma según el símbolo que esté más cerca, siguiendo las fronteras de decisión de los gráficos. Sin embargo, aún así se puede cometer un error. Esto sucede cuando el ruido genera que el símbolo salga de su región de decisión. La probabilidad de error se puede calcular entonces a partir de la cota de unión de fronteras, que considera que la probabilidad de considerar el símbolo j, cuando se transmite el símbolo i es $Q\left(\frac{d}{2\sigma}\right)$. Donde d es la distancia entre símbolos, $\sigma = N_0/2$ es la desviación estándar del ruido blanco gaussiano y Q la probabilidad asociada a una variable gaussiana normal estándar. De esta forma:

$$P_e \leq (M-1)Q\left(\frac{d}{2\sigma}\right)$$

A esta cota se le puede agregar otra restricción. Aquellos vecinos que están muy lejos del símbolo que se transmite casi no influyen en la probabilidad de error. Con d muy grande, $Q\left(\frac{d}{2\sigma}\right)$ es muy pequeño. En vez de

tomar todos los símbolos $M-1$ se toman $\langle N_e \rangle$ símbolos, que son los vecinos más cercanos. La cota de unión de fronteras por vecinos más cercanos está dada por:

$$P_e \leq \ \langle N_e \rangle Q\left(\frac{d}{2\sigma}\right)$$

Donde $\langle N_e \rangle = \sum_{i=0}^{M-1} p_i N_i$ con N_i la media ponderada de la cantidad de vecinos. En el caso de símbolos equiprobables se tiene:

$$\langle N_e \rangle = \frac{1}{M}\sum_{i=0}^{M-1} N_i$$

Para cada esquema de modulación, la probabilidad se expresa en función de M, por lo que d se expresa en función de ϵ_b y M, resultando finalmente en lo colocado en la tabla. Esta probabilidad de error que se analizó es de símbolo, también se puede analizar la probabilidad de error de bit. En el caso de que se realice el etiquetamiento con el código de Gray, como sucede en el caso de ASK, PSK y QAM, se considera que un error de símbolo (que tiene k bits) implicaría error solo en un bit. Esto es lo que sucede cuando se confunde un símbolo con uno vecino. Por lo que la relación entre P_e y P_b está dada por:

$$P_b \simeq \frac{P_e}{\log_2(M)}$$

En el caso del esquema de modulación FSK, no se utiliza el código de Gray. Como se explicó previamente, la distancia entre todos los símbolos es la misma por lo que no se puede implementar que símbolos vecinos difieran en un bit. En este caso se considera:

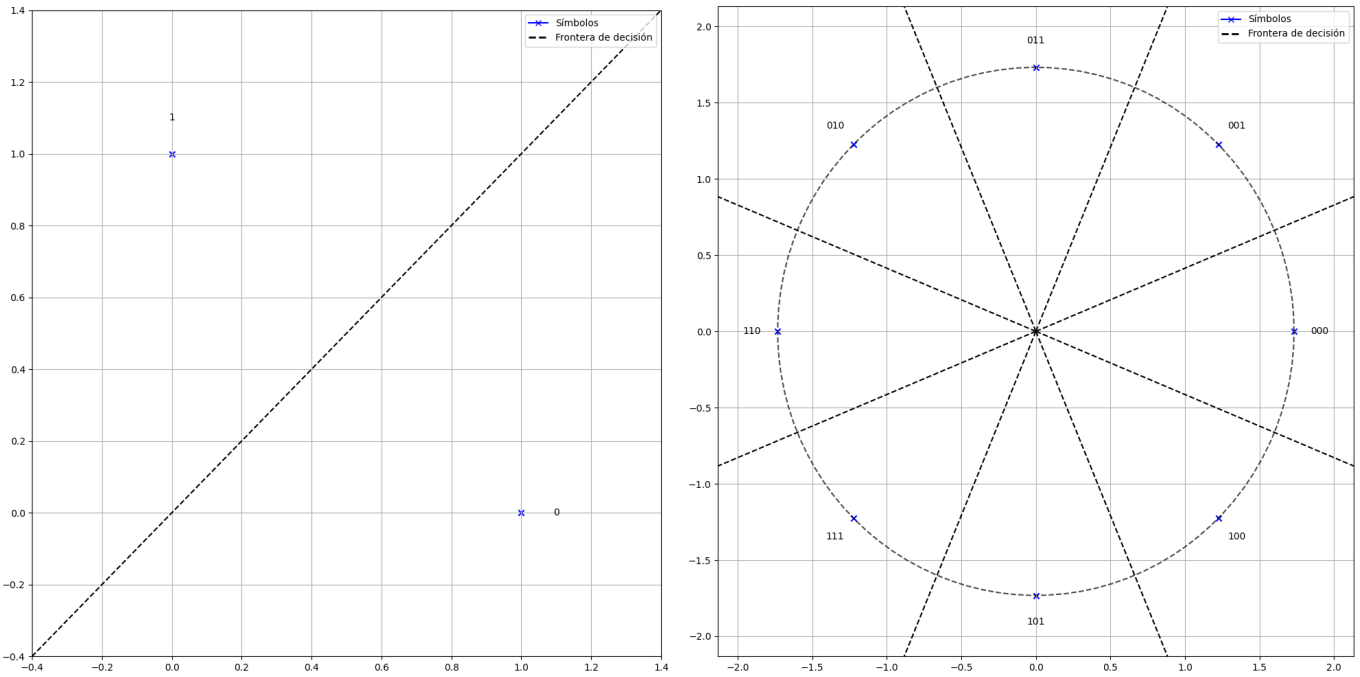
$$P_b \simeq \frac{M}{2(M-1)} P_e$$

	$\bar{\varepsilon}_b(d)$ [J]	$\bar{\varepsilon}_s(d)$ [J]	$P_b(M)$	$P_e(M)$
M-ASK	$\log_2 M \cdot \frac{d^2}{12}(M^2-1)$	$\bar{\varepsilon}_b(d)/\log_2 M$	$\frac{2(M-1)}{M\log_2 M} Q\left(\sqrt{\frac{6\varepsilon_b \log_2 M}{(M^2-1)N_0}}\right)$	$P_b(M)\cdot \log_2 M$
M-PSK	$\log_2 M \cdot \frac{d^2}{4\sin(\pi/M)}$	$\bar{\varepsilon}_b(d)/\log_2 M$	$\frac{2}{\log_2 M} Q\left(\sqrt{\frac{2\varepsilon_b \log_2 M}{N_0}}\ \sin(\pi/M)\right)$	$P_b(M)\cdot \log_2 M$
M-QAM	$\log_2 M \cdot \frac{d^2}{6}(M-1)$	$\bar{\varepsilon}_b(d)/\log_2 M$	$4\frac{1-1/\sqrt{M}}{\log_2 M} Q\left(\sqrt{\frac{3\varepsilon_b \log_2 M}{(M-1)N_0}}\right)$	$P_b(M)\cdot \log_2 M$
M-FSK	$\log_2 M \cdot \frac{d^2}{2}$	$\bar{\varepsilon}_b(d)/\log_2 M$	$\frac{M}{2} Q\left(\sqrt{\frac{\varepsilon_b \log_2 M}{N_0}}\right)$	$P_b(M)\cdot \frac{M-1}{M/2}$

Cuadro 2.4: Energía media por símbolo y por bit, y probabilidades de error de símbolo y bit para cada esquema de modulación.

En el caso de M-PSK, la $P_b(M)$ que aparece multiplicada por 2, solo es válido cuando $M \geq 4$. Si $M = 2$ ese factor en vez de ser 2, es 1.

Para los dos esquemas de modulación que se desarrollan en el trabajo, M puede tomar valores entre 2 y 16. Tomamos dos casos de cada esquema para graficar su constelación. En esta se observan los símbolos y las fronteras de decisión para cada caso.



(a) Constelación de la modulación 2-FSK.

(b) Constelación de la modulación 8-PSK.

Figura 2.2: Constelaciones de las modulaciones.

Se pide calcular la energía media de símbolo y de bit teórica para cada esquema de modulación. Como convención, se adopta una energía de bit para cada símbolo y en ambos esquemas unitaria, por lo que la energía de símbolo va a estar dada por:

$$\bar{\epsilon_s} = \log_2(M) = k$$

En el código, se puede estimar la energía media por símbolo directamente con la norma al cuadrado de la representación vectorial de cada símbolo. A partir de esta, se puede calcular la energía media por bit. Se realizan los cálculos para $M = 8$.

	M-PSK	M-FSK
$\bar{\epsilon_s}$ teórica [J]	3	3
$\bar{\epsilon_b}$ teórica [J]	1	1
$\bar{\epsilon_s}$ estimada [J]	3.00	3.00
$\bar{\epsilon_b}$ estimada [J]	1.00	1.00

Cuadro 2.5: Cálculo de energía media de bit y símbolo teórica y estimada para $M = 8$.

Los resultados teóricos y estimados son los mismos, considerando que al generar los vectores de los símbolos se utiliza justamente su energía teórica que se fija por convención. Luego, la norma al cuadrado de cada símbolo generado es justamente su energía, y para cada símbolo es la misma. Esto sucede para ambos esquemas de modulación, por lo que los resultados también son los mismos entre esquemas.

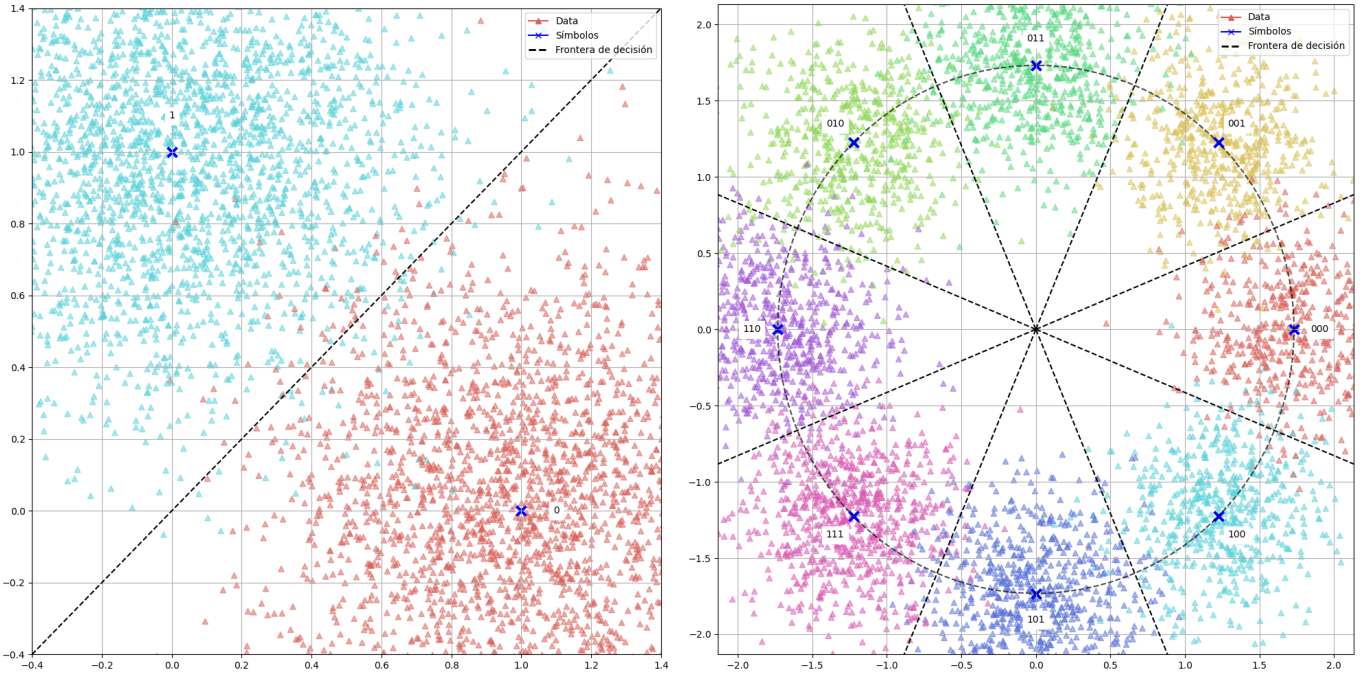
2.4. Módulo D: Efectos del canal

En este módulo se modelan los efectos del canal mediante ruido aditivo gaussiano blanco (AWGN), y se evalúa su impacto en la constelación a la entrada del receptor.

1. Se implementa una función que genera AWGN como una muestra gaussiana de media cero y varianza $\sigma^2 = N_0/2$ por dimensión.
2. Se implementa una función que aplica la atenuación y el AWGN a los símbolos modulados, utilizando los valores generados en los puntos anteriores.
3. Desde el programa principal, se grafica la constelación a la salida del canal, incluyendo regiones de decisión y etiquetado.

2.4. Resultados

El efecto del canal tiene una parte de ruido aditivo gaussiano blanco, y una parte de atenuación. Por simplicidad en la detección, como la atenuación no involucra cambios en la región de decisión, solo se implementó la primera parte. Los efectos del ruido aditivo se pueden observar en el gráfico de la constelación a la entrada del receptor. Se tomará un canal con $\mathrm{E_b}/N_0 = 6\,\mathrm{dB}$, con sus efectos reflejados en las configuraciones de 2-FSK y 8-PSK.



(a) Constelación de la modulación 2-FSK. (b) Constelación de la modulación 8-PSK.

Figura 2.3: Constelaciones de las modulaciones con efecto del canal.

Si fuera a considerarse la atenuación en el canal, se observarían estos mismos gráficos de constelación pero las muestras de los símbolos tendrían menos energía. Por cómo se definen las regiones de decisión, una disminución en la energía por símbolo hace más propensa a la muestra a cruzar de región cuando existe ruido. Para la generación de gráficos, se simula un texto como mensaje de fuente, que pasa por el bloque implementado de codificación de fuente y de modulación. Se le añade ruido y se toman 5000 puntos para graficar los símbolos que se reciben en los gráficos de constelación. Se identifican los símbolos originales por los colores de cada punto. De esta forma, se puede observar cómo la adición de ruido hace que los símbolos se dispersen alrededor del símbolo original, muchas veces cruzando las regiones de decisión hacia símbolos vecinos generando un error. En general, en PSK no se observan símbolos que se detecten como otros más lejanos a sus vecinos, que fue lo que se consideró para el cálculo de probabilidad de error. Todo esto se hace para una relación señal a ruido de $\mathrm{E_b}/N_0 = 6\,\mathrm{dB}$. Mientras mayor es la relación señal a ruido, mejor se distingue la señal y los puntos se concentrarían más alrededor de la posición de los símbolos originales, disminuyendo la probabilidad de error.

2.5. Módulo E: Codificación de canal (códigos lineales de bloques)

Entre el Codificador de fuente y el Modulador se incorpora el *Codificador de canal*; entre el Demodulador y el Decodificador de fuente, el *Decodificador de canal*.

Transmisor

1. Se implementa una función que, dado un mensaje de k bits, lo codifica en una palabra de código de longitud n mediante una matriz generadora G (código sistemático). Los valores de k, n y G son provistos por los Docentes y parametrizados desde el programa principal.

2. Se implementa una función que, dado un vector binario, lo agrupa en mensajes de k bits y genera el código lineal de bloques (longitud n) utilizando la función anterior.

Receptor

1) Se implementa una función que, dados k, n y G, obtiene la matriz de chequeo de paridad H.

2) Se implementa una función que, dada H, calcula la tabla de síndromes S.

3) Se implementa una función que, dadas H, S y una palabra de código de longitud n, calcula su síndrome, detecta y corrige (si es posible) los errores y devuelve la palabra corregida.

4) Se implementa una función que, dadas H y S, organiza el vector binario en palabras de n bits, detecta y corrige errores usando la función anterior, y decodifica.

5) Se implementa una función que calcula $d_{\text{mín}}$, la máxima cantidad de errores a detectar e y a corregir t.

2.5. Resultados

Para este trabajo se utilizó códigos de bloques lineales como método de detección y corrección de errores. Este método hace uso de una matriz generadora $G \in \{0,\ 1\}^{k\times n}$, donde $n = 15$ y $k = 5$, la cual esta dada por:

$$G = \left(\begin{array}{ccccc|ccccccccccc} 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right)$$

Esta matriz cumple la forma de una concatenación de una matriz identidad de dimensión k y una matriz $P \in \{0,\ 1\}^{k\times(n-k)}$, es decir $\left(\ \mathbb{I}_k \mid P\ \right)$.

Se toma un mensaje m de k bits, y multiplicándola por la matriz generadora $m \oplus G$ se obtiene el mensaje a transmitir. Partiendo de un ejemplo, en el cual se va visualizará el proceso de 3 bloques desde la codificación hasta la decodificación.

$$\begin{cases} m = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \end{bmatrix} \implies u = m \oplus G = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \\ m = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \end{bmatrix} \implies u = m \oplus G = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \\ m = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \end{bmatrix} \implies u = m \oplus G = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{cases}$$

Después de que estas palabras de código u hayan sido transmitidos, modulado y desmodulado. se obtienen los mensajes:

$$\begin{cases} \tilde{u} = \begin{bmatrix} 1 & 1 & 1 & 1 & \mathbf{1} & 1 & 1 & 0 & 0 & \mathbf{0} & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \\ \tilde{u} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & \mathbf{0} & 1 & 0 & 1 & \mathbf{1} & 0 & 0 & 1 & 0 \end{bmatrix} \\ \tilde{u} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{cases}$$

Para la decodificación se utiliza una matriz de paridad $H \in \{0,\ 1\}^{(n-k)\times n}$, donde cumple la propiedad que $G \oplus H^T = 0$, por lo que toma la forma $\left(\begin{array}{c|c} P^T & \mathbb{I}_{(n-k)} \end{array}\right)$, resultando:

$$H = \left(\begin{array}{ccccc|ccccccccccc} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right)$$

Esta propiedad permite diferenciar entre el mensaje originales y errores agregados, ya que el proceso propone lo siguiente

$$m \underset{\text{codificación}}{\Longrightarrow} m \oplus G \underset{\text{canal}}{\Longrightarrow} m \oplus G + e \underset{\text{decodificación}}{\Longrightarrow} (m \oplus G + e) \oplus H^T = \underbrace{m \oplus G \oplus H^T}_{=\ 0} + e \oplus H^T = e \oplus H^T$$

A partir de este resultado, si $\tilde{u} \oplus H^T = 0$ es que no ocurrió un error o que el error movió el mensaje a otro mensaje valido. Siguiendo el ejemplo, se obtiene:

$$\begin{cases} s = \tilde{u} \oplus H^T = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \\ s = \tilde{u} \oplus H^T = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \\ s = \tilde{u} \oplus H^T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{cases}$$

Como se mencionó anteriormente, en el caso que no existió un error es cuando el síndrome es nulo.

Existen 2^n posibles errores que pueden modificar el mensaje u y se tienen $2^{(n-k)}$ patrones que podemos reconocer con los bits de paridad, por lo que implica que no se pueden reconocer todos los errores para ser corregidos, es por esto que se utiliza una tabla de síndromes, la cual establece que error más probable genera un síndrome s. Se puede ver la tabla de síndromes en la Tabla 2.6.

Para el caso del ejemplo, se obtiene que los errores estimados son los siguientes:

$$\begin{cases} s = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \Longrightarrow \hat{e} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ s = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \Longrightarrow \hat{e} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \\ s = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \Longrightarrow \hat{e} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{cases}$$

Donde finalmente se puede aplicar la corrección obteniendo un mensaje estimado $\hat{m}$ que se obtiene como $\tilde{u} \oplus \hat{e}$, y finalmente se trunca el mensaje para obtener los k bits del mensaje original.

$$\begin{cases} \hat{m} = \tilde{u} \oplus \hat{e} = \begin{bmatrix} 1 & 1 & 1 & 1 & \mathbf{0} & 1 & 1 & 0 & 0 & \mathbf{1} & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \\ \hat{m} = \tilde{u} \oplus \hat{e} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & \mathbf{1} & 1 & 0 & 1 & \mathbf{0} & 0 & 0 & 1 & 0 \end{bmatrix} \\ \hat{m} = \tilde{u} \oplus \hat{e} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \end{bmatrix} \end{cases}$$

e															$e\ H^T$									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	1	1
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	1	1	1	1	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	1	0	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	1	0	1	1
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	0
...																								
1	0	0	1	0	1	1	0	0	0	0	0	0	1	0	1	1	1	0	1	0	1	0	1	1
0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1	1	1	1	1	0	0
0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	1	1	0	1	1	1	0
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	0	1
0	0	0	0	0	1	1	1	1	0	1	0	0	0	0	0	0	0	0	1	0	1	1	1	1

Cuadro 2.6: Tabla de síndromes.

La forma de caracterizar una código lineal es a través de la cantidad máxima de detección e y corrección t que logra la matriz generadora G. Estas se calculan a partir de una d_{min}, la cual representa la mínima distancia entre los mensajes después de haber sido multiplicados por G, es decir:

$$d_{min} = \min_{\substack{i,\ j \\ i \neq j}} d(m_i \oplus G,\ m_j \oplus G)$$

Esta distancia se puede expresar como $d(u,\ v) = W(u \oplus v)$, donde $W(\cdot)$ es el peso de Hamming, por lo que reescribiendo la distancia mínima como:

$$d_{min} = \min_{m \neq 0} W(m_i \oplus G) = 4$$

Finalmente, se puede calcular los valores de e y t de la siguiente forma:

$$\begin{cases} e = d_{min} - 1 = 3 \\ t = \left\lfloor \frac{d_{min}-1}{2} \right\rfloor = 1 \end{cases}$$

3. Análisis del sistema

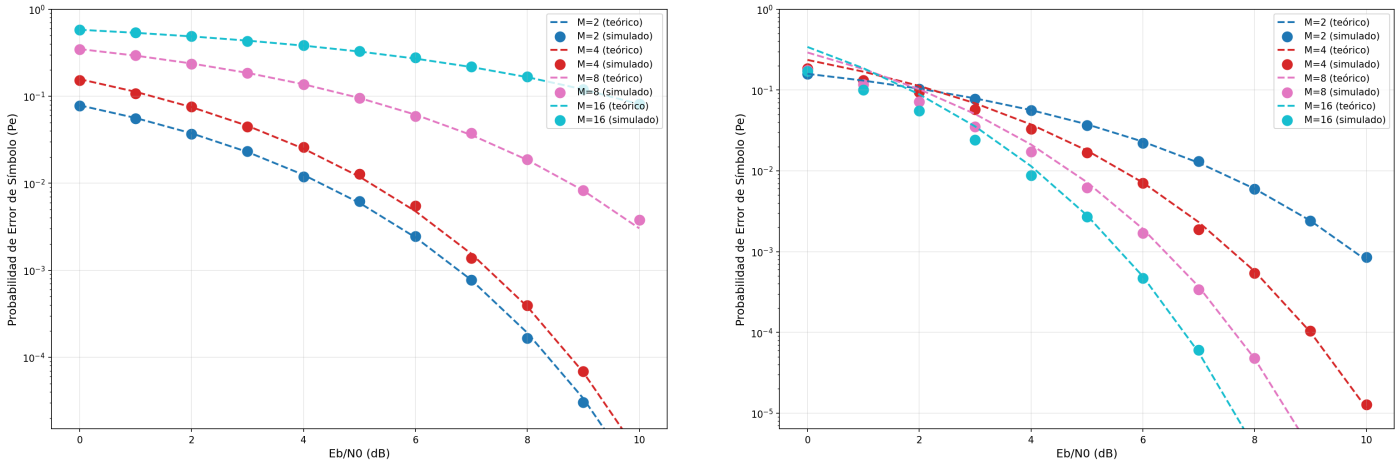
Con todos los módulos implementados, simular desempeño ante variaciones de parámetros.

1. Para cada modulación y valores de M propuestos, estimar y graficar P_e y P_b en función de $\mathrm{E_b}/N_0 \in [0, 10]\,\mathrm{dB}$ (paso 1 dB). Variar N_0 manteniendo $\mathrm{E_b}$ constante. **Deshabilitar** codificación/decodificación de canal en este ítem.

2. Repetir el punto anterior usando un código lineal de bloques (n, k) con matriz generadora G indicada por los Docentes.

3.1. Resultados

3.1. Probabilidad de Error de Símbolo (P_e) - Sin Codificación de Canal



(a) PSK (b) FSK

Figura 3.1: Probabilidad de error de símbolo P_e vs $\mathrm{E_b}/N_0$ para PSK y FSK con $M = \{2, 4, 8, 16\}$ (sin codificación de canal).

Utilizando la relación entre P_e, E_b/N_0 y M para cada esquema de modulación dados en la tabla 24, se grafica P_e vs E_b/N_0 para distintos M. Además de los gráficos teóricos, se agregan las muestras estimadas calculando la probabilidad de error de símbolo como la proporción entre símbolos incorrectamente detectados y todos los símbolos transmitidos. Para probabilidades de error muy pequeñas, el método de estimación por Monte Carlo no resulta muy representativo. El problema surge en que el número de eventos observados muchas veces no es suficiente para obtener una estimación confiable. Se requerirían muchísimas muestras para acercarse a la realidad, lo que se vuelve computacionalmente pesado. De la secuencia de bits que llegan del bloque de codificación de fuente, se toma una porción de bits para generar los gráficos. Con estos se generan M símbolos, los cuales luego se comparan con los símbolos recuperados en el demodulador. La proporción de símbolos erróneos respecto del total de símbolos será la probabilidad estimada. Sin embargo, se fija una cota. Si la cantidad de errores es menor a 100, se toman el doble de bits, es decir, de muestras para estimar la probabilidad y obtener un mejor resultado a partir del método de Monte Carlo. Cuando ya no haya más bits provenientes del archivo para usar, se opta por no graficar esta estimación, considerando que se necesitarían demasiadas muestras para llegar a un resultado razonable dado las limitaciones del método de estimación.

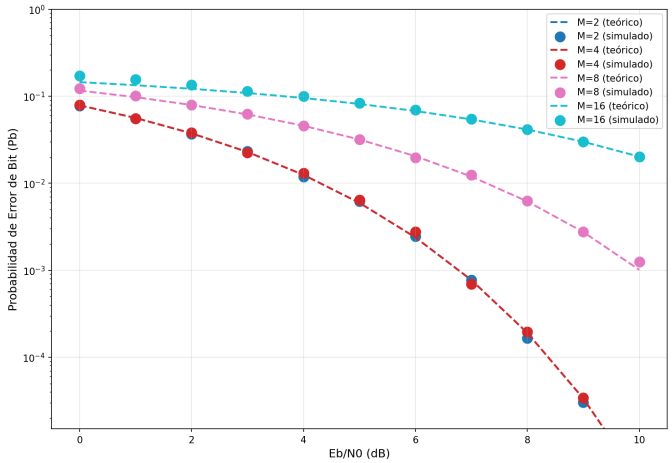
Hay varias cuestiones que se pueden analizar de estos gráficos. Lo primero que se puede observar es que en ambos esquemas de modulación, la probabilidad de error disminuye con el aumento de la relación señal a ruido. Esto es esperable, dado que para un E_b fijo, a medida que N_0 disminuye, el efecto del ruido se hace cada vez más despreciable. Sin embargo, en general, las muestras estimadas sigue la forma de la teórica, validando las fórmulas implementadas.

En el caso de M-PSK, hay dos efectos que se dan con el aumento de M en la probabilidad de error de símbolo, uno positivo y otro negativo. Por un lado, cuando M aumenta, manteniendo constante E_b, aumenta E_s. En el diagrama de constelación de M-PSK se observarían los símbolos cada vez más alejados del origen. Este aumento en E_s aumenta la distancia entre símbolos y por ende disminuye la probabilidad de error de

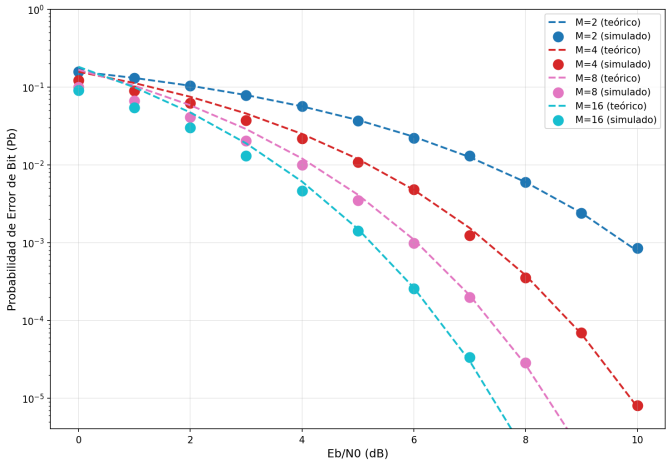
símbolo. Sin embargo, esta distancia no solo depende de la energía de símbolo sino de la distancia angular entre ellos dada por $\frac{2\pi}{M}$, que disminuye con M. En el caso de $M = 2$ y $M = 4$ estos dos efectos se compensan por el aumento de dimensión y la distancia de símbolos es la misma. La diferencia entre las probabilidad de error de símbolo provienen de considerar un vecino en el caso de $M = 2$ y 2 vecinos en el caso de $M = 4$ para la cota de unión de fronteras por vecinos más cercanos. Para M mayores, el efecto predominante es la disminución de la distancia entre símbolos, la región de decisión se hace cada vez más angosta y los símbolos recibidos con ruido son más propensos a equivocarse con vecinos. Con el aumento de M, aumenta la probabilidad de error de símbolo.

En el caso de M-FSK también se dan dos efectos con el aumento de M, uno positivo y uno negativo. La distancia entre símbolos solo depende de E_s, que aumenta con M. Por lo que la distancia entre símbolo y por ende la probabilidad de error de símbolo aumenta con el aumento de M. En M-FSK las distancias entre todos los símbolos son las mismas, por lo que también lo será la probabilidad de confundir un símbolo con uno vecino. Sin embargo, mientras más vecinos tomamos, tenemos que considerar cada vez más casos en donde el símbolo pueda ser erróneo, porque cada vez hay más vecinos con los que se puede confundir. Esto hace que la probabilidad de error aumente con M. Analíticamente, con el aumento de M, el argumento de $Q(\cdot)$ aumenta $\propto \sqrt{log_2 M}$, lo que genera una disminución de $Q(\cdot)$. Sin embargo, el término $M - 1$ que lo multiplica aumenta con M. En el gráfico, se puede observar que para relaciones señal a ruido bajas, mientras mayor es M, mayor es P_e. Para $E_b/N_0 \gtrapprox 2$ dB, esta situación se invierte y la probabilidad de error de símbolo disminuye con el aumento de M, predominando el efecto positivo previamente mencionado. Esto se observa principalmente en la curva teórica, dado que para E_b/N_0 bajos las muestras estimadas para cada M se encuentran prácticamente superpuestas.

3.1. Probabilidad de Error de Bit (P_b) - Sin Codificación de Canal



(a) PSK

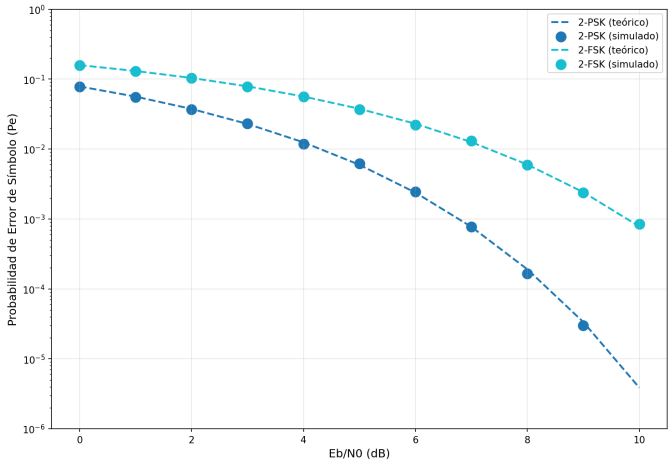


(b) FSK

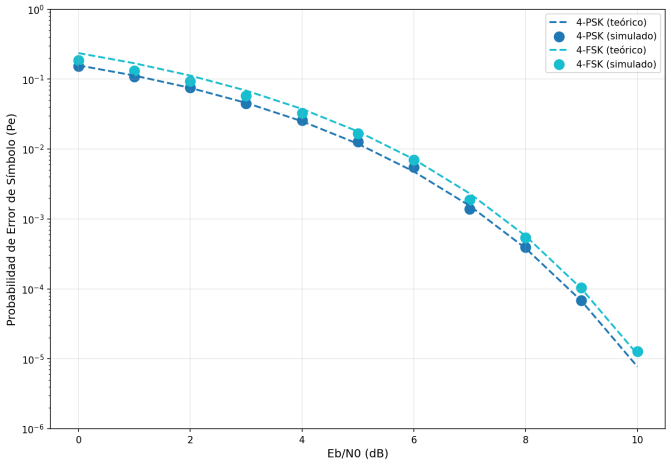
Figura 3.2: Probabilidad de error de bit P_b vs $\mathrm{E_b}/N_0$ para PSK y FSK con $M = \{2, 4, 8, 16\}$ (sin codificación de canal).

La probabilidad de error de bit sigue un comportamiento similar a la probabilidad de error de símbolo. En general, $P_b \leq P_e$, con igualdad en el caso binario ($M = 2$). La relación entre P_b y P_e cambia según el esquema de modulación debido a que cambia el código utilizado en el etiquetamiento, como bien se explicó previamente. En M-PSK, al utilizar el código de Gray se toma que un error de símbolo se da con error solamente en un bit. En el caso de M-FSK, puede haber más errores de bits en un error de símbolo, por lo que para un mismo valor de M y de P_e, la probabilidad de error de bit es mayor.

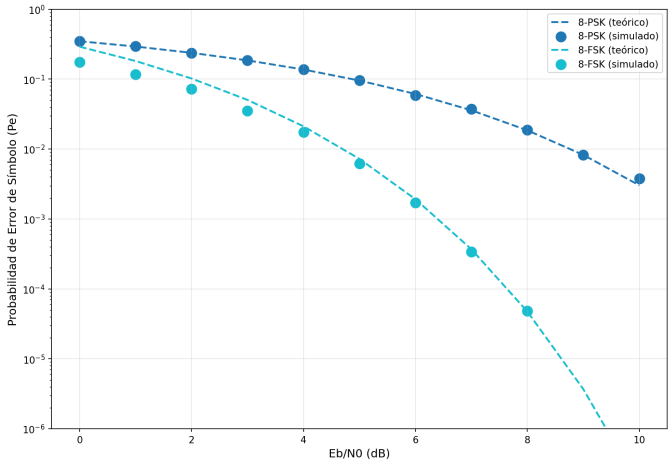
3.1. Comparación entre Modulaciones Eficientes en Ancho de Banda vs Energía



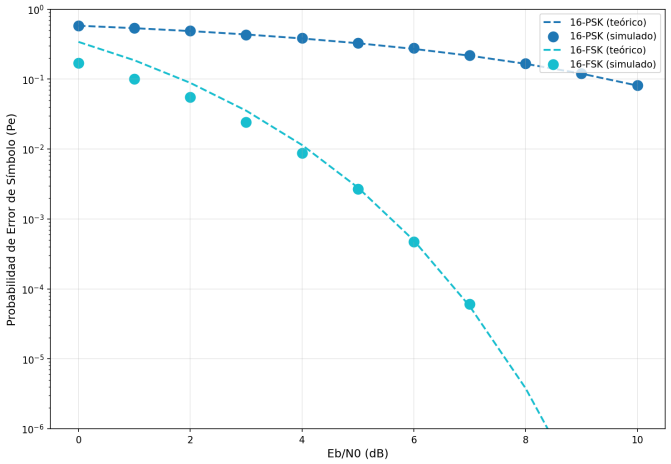
(a) $M = 2$



(b) $M = 4$



(c) $M = 8$



(d) $M = 16$

Figura 3.3: Comparación entre PSK (eficiente en ancho de banda) y FSK (eficiente en energía) para diferentes valores de M.

Se generan gráficos que comparan para cada M la curva de P_e vs E_b/N_0 entre los dos esquemas de modulación que se analizan en el trabajo. Para $M = 2$, se observa que la probabilidad de error en el caso de PSK es menor que en FSK. Justo en este caso, los símbolos son antipodales, por lo que tiene sentido que minimicen la probabilidad de error en comparación con el caso de FSK donde los símbolos son ortogonales debido a la correlación entre símbolos.

Para el caso de $M = 4$ el comportamiento entre ambos esquemas es bastante similar, siendo la probabilidad de error en PSK todavía un poco menor que en FSK.

Es interesante ver qué pasa cuando M aumenta porque la situación se revierte, y FSK pasa a tener menor probabilidad de error. Esto es justamente lo que se comentó antes, con M-FSK aumenta la cantidad de dimensiones y los símbolos están más espaciados entre sí. Sin embargo, más dimensiones implican más frecuencias, recordando que cada eje está dado por una señal coseno de distinta frecuencia que conforma una base ortonormal del espacio. Por eso, además, se exige una separación mínima de frecuencias para que las señales sean ortogonales. El ancho de banda entonces crece proporcionalmente con M. Esto no sucede en el caso de M-PSK porque la frecuencia de la portadora se mantiene fija, por eso es una modulación eficiente en ancho de banda, como también lo son M-ASK y M-QAM. Por más que utilice más ancho de banda, para un mayor M, M-FSK necesita menos energía que M-FSK para lograr probabilidades de error bajas, lo que lo hace un esquema de modulación eficiente en energía. Esto se puede observar en los gráficos, donde las curvas de M-FSK caen más rápido que las de M-PSK.

3.1. Efecto de la Codificación de Canal

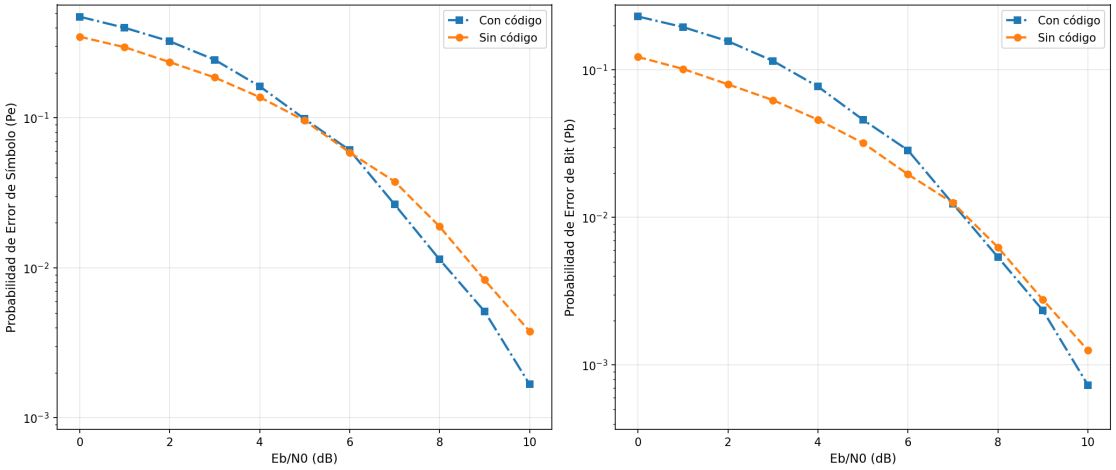


Figura 3.4: Comparación de probabilidad de error de símbolo P_e y probabilidad de error de bit P_b vs $\mathrm{E_b}/N_0$ con y sin codificación de canal para PSK con $M = 8$.

Se evaluó el efecto de la codificación de canal utilizando un código lineal de bloques $(15, 5)$ con la modulación PSK y $M = 8$. Cada 5 bits agregaremos 10 bits de redundancia, y esas palabras de código de canal pasarán por el bloque modulador. Aquí, se agrupan los bits de a k y se generan los símbolos que se envían a través del canal, con un ruido aditivo blanco y gaussiano. Acá tendremos símbolos que son informativos y otros que se conformaron con bits de paridad. Una vez que llegan al demodulador, se toma una decisión para cada símbolo recibido, buscando recuperar la secuencias de bits que habrían llegado al modulador. En el decodificador de canal, se analizará si la palabra es válida y en el caso que no lo sea la corrige siguiendo la tabla de síndromes. Luego, descarta los 10 bits de redundancia. En este punto, con esta secuencia de bits, volvemos a realizar el etiquetamiento para volver a hablar de símbolos y observar cómo cambia la probabilidad de error de símbolo y de bit una vez que el bloque realiza la corrección de cada palabra. En ambos casos, se grafican las curvas estimadas a partir del método de Monte Carlo. Por más que no sea un parámetro en el trabajo, es importante considerar que si la tasa de transmisión de bits no codificados (tasa binaria útil) se busca mantener constante, la tasa de bits codificados aumentará en razón de n/k, lo que va a implicar mayor ancho de banda. A igual potencia del transmisor y si la energía por bit sin codificar se mantiene constante, la energía por bit codificado va a disminuir porque ahora tendremos la misma energía repartida entre más bits codificados. $E_c = E_b \frac{k}{n}$. Para realizar el gráfico en función de E_b/N_0 hay que hacer un escalamiento para el caso de utilizar codificación de canal, para tomar la energía por bit codificado.

Se observa en ambos gráficos dos regiones de análisis definidos por un umbral, distinto en cada caso. El umbral para la probabilidad de error de símbolo se encuentra en $\mathrm{E_b}/N_0 = 6\,\mathrm{dB}$, y el umbral para la probabilidad de error de bit se encuentra en $\mathrm{E_b}/N_0 = 7\,\mathrm{dB}$, aproximadamente. Para valores debajo del umbral, en el caso de Eb/N_0 bajos, la probabilidad de error de símbolo y de bit es menor sin codificación de canal que con. Cuando consideramos mucho ruido, sabemos que la probabilidad de que un símbolo se confunda con uno vecino es muy alta y en el caso de PSK esto implicaría un error de un bit. Nuestra implementación de codificación de canal permite la corrección de un bit pero en una palabra de código de 15 bits, que en realidad estaría conformada por varios símbolos. Cuando aparecen más errores que los que el bloque es capaz de corregir de todas formas realiza una corrección utilizando la tabla de síndromes, pero no asegura que sea la correcta. Muchas veces no solo no puede corregir errores sino que incluso los agrega. Pensemos que tenemos una palabra de código, donde los primeros 5 bits son informativos. La palabra de código que recibe el decodificador tiene esos mismos 5 bits pero tiene errores en los bits de redundancia, por lo que intenta corregirlo pero se equivoca y cambia algunos de los primeros 5 bits que sí estaban bien. Cuando lo etiquetamos y agarramos, en este caso, los primeros 3 bits para conformar un símbolo, el símbolo que obtenemos va a ser incorrecto y ese error no se introdujo por el ruido sino por el intento del bloque decodificador de corregir toda la palabra de código.

En la región de análisis correspondiente a valores de E_b/N_0 mayores al umbral, se observa cómo habilitar el bloque de codificación y decodificación de canal disminuye la probabilidad de error de símbolo y de bit. En este caso, el ruido ya no genera tanta dispersión y equivocarse con un símbolo vecino es menos probable. Habrán

errores en algunos símbolos pero se podrían conformar palabras con errores únicamente en un bit, para los cuales el método de codificación podrá recuperar la palabra de código original. En estos casos podrá entonces detectar y corregir errores, disminuyendo la probabilidad de error. Incluso cuando haya más errores que un solo bit en la palabra de código, el bloque de codificación podrá corregirlo pero no lo asegura. También puede seguir cometiendo errores, e incluso en el caso de probabilidad de bit se necesita un mayor E_b/N_0 para que sea mejor utilizar codificación de canal que no utilizarla, en comparación con la probabilidad de símbolo. Nosotros asociábamos que un error de símbolo, en el caso de PSK, se corresponde con un error de bit. El problema es que el bloque de codificación de canal corrige de a palabras de código y cuando intente corregir y se equivoque puede que ese error de símbolo implique errores en más bits, no solo uno. Por esto se necesita una menor relación señal a ruido para que la codificación disminuya la probabilidad de bit respecto a no utilizarla. Necesita que haya muy pocos errores introducidos por el ruido, para que en la mayoría de los casos haya un error de bit por palabra de código y no se introduzcan tantos errores cuando el codificador se equivoca.

3.1. Efecto en el texto ingresado

En la sección anterior, se analizó la probabilidad de error y de símbolo en función de E_b/N_0 habilitando y deshabilitando la codificación/decodificación del canal. Buscamos visualizar el efecto del bloque de codificación de canal de otra forma: con un texto de prueba. Se agrega un texto de prueba al principio del texto de entrada que se utilizó a lo largo del trabajo que dice:

```
Esto es una linea de prueba para ver el resultado obtenido despues de todo este proceso.
```

Este texto pasará por todos los bloques del sistema, utilizando un esquema de modulación 8-PSK y habilitando y deshabilitando la codificación de canal. Luego se observa el texto generado a la salida del sistema. Este proceso se repite para distintos valores de E_b/N_0, estos siendo de 4 dB, 6 dB y 8 dB. Esta es una forma visual de evaluar, según la relación señal a ruido, como actúa el codificador de canal en la corrección de errores que se generan a lo largo del proceso. Esperaríamos que a mayor E_b/N_0, el texto resulte más legible con el bloque codificador que sin este.

E_b/N_0	Sin bloque codificador	Con bloque codificador
4 dB	Esto es inatiiteinolpvnanl para dtr en retultas,Nbtenido despues de toduleso arrontso. L	Esto es una linea de loceba para rtnlel resulsldo obtenidoamraruOoltodo este procesoucre
6 dB	Esto es una linea de prueba hpa ver el resultado naleera utmndm es de todo este proceso	Esto es una lideinolprueblepara ver elinral,ado obtenido despues de todo este proceso. r
8 dB	Esto es una linea de prueba para ver el reso tado obtenir, despues de e,do este proceso.	Esto es una linea de prueba para ver el resultado obtenido despues de todo este proceso.

Cuadro 3.1: Comparación de texto como resultado del proceso sin y con codificación de canal.

Para el caso de $E_b/N_0 = 8$ dB, recuperamos con el bloque codificador directamente el mensaje original, a diferencia de sin utilizarlo. Para E_b/N_0 menores se presentan errores tanto utilizando como sin utilizar el bloque codificador de canal. Cambian según el caso los errores que se cometen, lo que hace más legible solo algunas partes del texto según el caso pero no completamente. Es importante considerar también que Huffman es bastante frágil a errores de bit. Si tomamos el ejemplo analizado en la sección 2.2, y el texto de entrada es ADBADEDBBDD, lo que debería llegar al decodificador de fuente sin errores es la secuencia 11101011101100101000. Pero digamos que se introducen 2 errores de bit y llega **0**1100**0**11101100101000. El decodificador de fuente en este caso devuelve DEBDDADEDBBDD. Como es un código de longitud variable, el error en un bit puede hacer que arrastre algunos errores de caracteres y resulta incluso en una texto de longitud distinta que el original. Esto no pasaría, por ejemplo, utilizando un código de longitud fija como ASCII. Con un error de bit, tendría un error de caracter pero no lo va a arrastrar.

4. Conclusiones

Lo primero que se desarrolló en el trabajo fue la codificación y decodificación de fuente. Para ello se implementó el algoritmo de Huffman, el cual permite construir un código óptimo en longitud promedio cuando se conoce la distribución de probabilidades de los símbolos de la fuente.

La idea central del método es asignar palabras de código más cortas a los símbolos más probables, y palabras más largas a los símbolos menos frecuentes, de manera tal que la longitud promedio por símbolo resulte mínima. A diferencia de esquemas de longitud fija, como ASCII, donde cada símbolo tiene asociada una palabra de código de longitud fija, en Huffman la longitud de cada código es variable y adaptativa a la distribución estadística de la fuente.

Tal como se esperaba, se logró un mayor rendimiento utilizando la codificación por Huffman, conocido como el cociente entre la longitud promedio por símbolo y la longitud mínima teórica fijada por el primer teorema de Shannon. En este bloque justamente se busca reducir la tasa de bits a transmitir y disminuir el ancho de banda necesario, por lo que buscamos una menor longitud promedio (y mandar por ende menos bits). Sin embargo, hay que tener en cuenta que el receptor debe recibir el árbol de Huffman para poder decodificar y recuperar el mensaje original. Esto se transmite solo una vez, y podría ser despreciable su costo con el tamaño de datos, comparando con la utilización de un código de longitud fija como ASCII donde este paso no sería necesario pero tiene una eficiencia muchísimo menor.

Luego de la codificación de fuente, el mensaje ahora expresado como secuencia de bits con las palabras de código asociadas a cada símbolo de fuente llega al bloque de codificación de canal. En este bloque se agregan redundancias de forma controlada para la detección y corrección de bits. Esto se implementa utilizando códigos de bloques lineales, considerando que se reciben mensajes de $k = 5$ bits que se codifican en $n = 15$ bits. Se calculó la distancia mínima, que resultó ser igual a 4, y se concluyó que el código permite detectar 3 errores de bits y corregir 1 solo. Se comprobó la eficiencia de la detección y corrección de errores observando que la probabilidad de error de símbolo y bit disminuye implementando la codificación de canal respecto de cuando no se implementa.

Luego del bloque de codificación de canal, se llega al bloque de modulación. Los bits se agrupan de a k según la cantidad M de símbolos de canal que se utilicen. A cada símbolo de canal se le asigna entonces una secuencia de bits, la cual depende del esquema de modulación. Se analizó que en M-PSK el etiquetamiento se realiza utilizando códigos de Gray y en M-FSK códigos binarios. Se analizó que el etiquetamiento óptimo de símbolos en M-PSK se realiza mediante un código de Gray porque este minimiza la probabilidad de error de bit por confusión entre símbolos vecinos. Esto se logra porque en el código de Gray dos símbolos adyacentes difieren en un bit. Esto no se puede aprovechar en el esquema de modulación M-FSK porque todos los símbolos están a igual distancia, son ortogonales y son todos vecinos entre sí. No se puede lograr un etiquetamiento en el cual todos los vecinos difieran en un bit excepto que se usen M bits para representar cada símbolo. Por esto, se utilizan códigos binarios para M-FSK. Un error de símbolo podría implicar un error de solo un bit o hasta de los k bits, porque podría confundirse con cualquier vecino (que son todos los símbolos).

Para el bloque de modulación se observaron los efectos de M y de E_b/N_0 en la probabilidad de error y de símbolo en cada esquema de modulación. Se pudo analizar que a medida que M aumenta, en M-FSK disminuye la probabilidad de error, a diferencia de lo que sucede con M-PSK. M-FSK logra probabilidades de error pequeñas dada una menor relación señal a ruido que M-PSK, demostrando ser un esquema eficiente energéticamente pero a costo de utilizar un mayor ancho de banda. Asimismo, en M-PSK, la frecuencia de la señal es fija, no cambia con M, por esto es eficiente en ancho de banda. Sin embargo, a medida que M aumenta, la probabilidad de error aumenta y necesita mucha energía para lograr valores de probabilidad de error pequeñas.